Real Time Systems and Industrial Applications
Project Proposals
a.a. 2025/26

Assignment and preparation rules

Projects assignment: Projects are assigned to groups of maximum 3 students. Each group must contact me by June 5th 2026 with the composition of the group (students name and matricola), nickname of the group, and three preferences for projects in order (e.g., A2, A1, I2). The assigned project will be communicated by maximum June 9th. We have both Analysis and Implementation projects you can choose from.

Meetings rule with tutors: A tutor will be assigned for each project. Max 3 meetings are allowed, to be organized with the tutor, per project:

- first meeting: presentation of the project and its objectives by the tutor, gathering of necessary material
- second meeting: presentation of the results from the first meeting, discuss clarification, steer for the final meeting
- third meeting: presentation of the results from the second meeting, discussion on how to organize the work for the final presentation

Project submission and presentation rule:
The project description and results must be reported in a brief report, scientific paper style, with introduction, objectives, presentation of design and experimental results. Max 10 pages.
Also a power point presentation must be prepared to present the results at the exam (max 10 minutes presentation per group participant, e.g., 20 minutes total for a group of two students).
The project report and presentation must be submitted to me via email or Teams **2 days before the exam date**. If required by the students, the oral exam can be postponed to a different date (also for the different member of the group, however the project presentation must be performed by the whole group in the same exam date).

Analysis Projects

A1) EdgeAI real-time interference: a performance study

Edge devices are inherently multi-tenant: in any realistic deployment a language model never runs alone, but shares CPU, cache, memory, bus, and accelerator with other processes (system services, telemetry, pre/post-processing pipelines). Interference has been studied in datacenters and on GPU servers, but on edge Single Board Computers, with small caches, shared memory, general-purpose schedulers, and non-virtualizable accelerators, the phenomenon is qualitatively different and poorly characterized for language models, whose compute and memory access patterns are distinctive (compute-bound prefill, memory-bound decode). Moreover, sensitivity to interference depends strongly on model scale: a 1–3 B SLM and a 7–13 B LLM respond very differently to resource saturation.

This project produces an experimental characterization of interference for language-model inference using stress-ng, whose stressors are organized into classes mapped onto the physical subsystems of the kernel (CPU, memory, cache, I/O, scheduler, OS, interrupt) precisely the analysis dimension the project explores. No errors are injected: the system keeps working correctly, and what changes is the level of pressure on shared resources.

Tasks for Students

- Study the inference runtime (llama.cpp) on the assigned board and establish a baseline characterization of inference at rest for each model and task.
- Map the stress-ng stressor classes onto the physical resources of the board and select 2–3 representative stressors per class (CPU: cpu, matrix, fft; memory: vm, stream, memrate; cache: cache, l1cache; I/O: iomix, hdd; scheduler/OS: sched, switch, fork; interrupt: timer, sigq).
- Design an experimental matrix that, for each stressor class, varies the intensity (number of workers, working-set size scaled on the board cache, duration) and measures the impact on inference along two dimensions: performance (per-token latency, time-to-first-token, throughput, jitter, temperature, throttling) and quality (truncation on timeout, incomplete output, decoding interruptions).
- Use objective ground-truth tasks (multi-digit arithmetic, multiple-choice QA on MMLU/ARC-Easy subsets, structured JSON extraction) to enable objective degradation metrics (accuracy drop, truncation rate). Use greedy decoding with fixed seeds to isolate inference variance from interference-induced variance.
- Correlate application-level degradation with hardware phenomena using system metrics (perf counters, /proc, thermal sensors): cache miss rate, branch mispredictions, frequency throttling.

- Build per-class degradation curves and resource-sensitivity maps for the board, identifying which resource, once saturated, degrades inference the most.
- Optional, for larger groups) Repeat the study on a second board of a different architecture for a cross-hardware comparison, and, where the platform allows it, run an LLM (Llama-3.1-8B or Mistral-7B in Q4) alongside the SLMs to compare interference sensitivity across model scale.

What the Students Will Learn

- The internals of resource contention on shared-memory edge platforms and how it maps onto kernel subsystems.
- How to drive stress-ng systematically by stressor class and read hardware performance counters.
- The distinctive compute/memory profile of language-model inference (prefill vs decode) and how it interacts with each class of resource pressure.
- Methodology to design a reproducible experimental campaign and present results with appropriate statistical visualizations.

Required Tools

- OrangePi 5 plus, NVIDIA Jetson Orin, Beaglebon, AMD ZCUs (104, 102, Krya)
- The llama.cpp inference runtime, built with the backend appropriate for the board.
- SLM models in GGUF (SmolLM2-1.7B, Qwen2.5-1.5B, Llama-3.2-1B); optionally LLM models (Llama-3.1-8B or Mistral-7B) where the platform allows.
- stress-ng, perf, and standard Linux system-monitoring tools.

Expected Deliverables

- An automated stress-testing and benchmarking framework based on stress-ng.
- The dataset of raw and aggregated results.
- An experimental report with per-class degradation analysis and, where applicable, cross-board and cross-scale (SLM vs LLM) comparison, including visualizations.

Note

Team size: 1–3 people (project scope scales with team size; the optional tasks are expected for larger groups, as agreed with the tutor). Duration: 4 weeks.

A2) Investigating Real-Time Communication Between Asymmetric Processors with OpenAMP

Modern SoCs often feature asymmetric multiprocessing (AMP) environments, where heterogeneous cores (e.g., ARM Cortex-A and Cortex-R/Cortex-M) coexist and need to communicate. The **OpenAMP framework** provides a standardized way to enable this inter-processor communication (IPC), traditionally via **RPMsg** over shared memory. Recently, OpenAMP has begun exploring **VirtIO**-based mechanisms—"AMP VirtIO"—which should remove the real-time limitations of the traditional VirtIO developed for KVM.

In this project, the student will investigate the current state and capabilities of the OpenAMP framework, focusing specifically on its **VirtIO-based communication mechanisms**. This includes analyzing how **AMP VirtIO** is implemented, evaluating the behavior through existing OpenAMP demos with **Zephyr RTOS**, which is used as the reference RTOS for real-time processing on the secondary core.

Tasks for Students

- Watch the official OpenAMP tutorial videos to get an overview: https://www.youtube.com/watch?v=R_5DOIvo7tI https://www.youtube.com/watch?v=kjoQ2Gidbuo
- Set up the OpenAMP development environment by following the official documentation https://openamp.readthedocs.io/en/latest/index.html
- Replicate and run the **Hypervisorless VirtIO demo** from OpenAMP: https://openamp.readthedocs.io/en/latest/demos/hvl_virtio.html
- Observe and document how messages are exchanged between the two processors in the demo.
- Identify where VirtIO is used in the code and what role it plays in the communication.
- Try modifying a simple message or behavior in the demo to see how both sides react.
- Summarize what features of VirtIO are used and what is still missing or limited.

What the Students Will Learn

- The basics of **asymmetric multiprocessing (AMP)** and why it's important in embedded systems.
- How the **OpenAMP framework** enables communication between different types of

- The purpose and structure of **VirtIO** in communication between virtual or real machines.
- How to set up and run real-world demos using **Zephyr RTOS** and embedded development tools.
- How to observe, modify, and understand **inter-processor message passing** in a real-time system.

Note

Team size: 1–3 people (project scope scales with team size; the optional tasks are expected for larger groups, as agreed with the tutor). Duration: 4 weeks.

A3) Investigating Kubernetes performance bottlenecks: how much does the request processing affect the orchestration latency?

Kubernetes (a.k.a. K8s) is a well-known container orchestrator, vastly used in modern cloud environments. The main entity is the cluster, a set of computational resources and storage units. The control plane orchestrates the workloads to run in the clusters on worker nodes.

The kube-apiserver allows K8s internals to communicate with one another and lets users interact with the cluster. User interactions span from data retrieval to resource manipulation. The usual way to create or modify a K8s resource is to encode its specification in a manifest file (usually, a ". yaml" file). Multiple users may concurrently send multiple manifest files to the cluster. A single user can even send multiple concurrent manifests or also encode multiple resources in a single file.

The students are requested to investigate the overhead introduced by the request processing under different load conditions and manifest configurations. Students should leverage the Kubernetes monitoring data to retrieve the information required to characterize the overhead.

Suggestion: The OpenTelemetry Framework (a.k.a. OTel) is well-integrated in the K8s platform. It can be leveraged to retrieve appropriate metrics, logs, and traces/spans automatically produced by the K8s internals.

Tasks for Students

Students should be able to answer the following research questions:

1. How does K8s behave with manifests of different resources? (e.g., Pod, Deployment, Services, etc)

2. How does K8s behave with manifests of the same resource, each representing a different possible configuration? (e.g., Pods with resource requests and limits, Pods with a security context, Pod with minimal specification, etc.
3. How does K8s behave when multiple resources are encoded in the same manifest?
4. How does Kubernetes behave with multiple manifests to process?

What the Students Will Learn

- Dealing with real, state-of-the-art monitoring tools and container platforms and technologies.
- Identifying the most accurate, valid and appropriate monitoring data and techniques.
- Designing a proper experimental campaign.

Required Tools

- A PC with at least two VMs, one for the control plane and one for the worker node, both with sufficient memory and CPUs.
- In alternative, tools like “Minikube” can be leveraged to create and simulate a simple cluster in a single VM or Linux-based host.
- As a last alternative, a single VM or a single Linux-based host can be used for a cluster including only the control plane. It should be configured to let Pods execute on it.

Expected Deliverables

- Source code and configuration files.
- Measurements answering the aforementioned research questions.
- Each measurement should be repeated multiple times (10-30) and may consist of a single metric identified in the K8s monitored data, or a derived measurement from the available monitoring sources.

Notes

Students should be aware that the lack of evidence to answer the aforementioned research questions is also a valid finding, if motivated by proper analysis and experimental campaign. Team size: 2-3 people. Duration: 4-6 weeks

A4) Evaluating the impact of code instrumentation on SLOs compliance of Real-Time task sets.

Real-Time systems are characterized by their logic and temporal behaviour. These tasks have to meet their real-time constraints in order to provide a reliable service and avoid system performance degradation, and dangerous scenarios in hard real-time environments.

A key role is played by the schedulability checkers, both online and offline, that ensure

that a given task set is schedulable, with each task compliant to its timely behaviour. The schedulability analysis often requires system metrics, like task CPU utilization. WCET, blocking times and all contributors to task execution times are obtained in the design phase.

The students are asked to identify state-of-the-art real-time and open-source programs (with estimated deadlines) and establish their WCET (on the platform the students are working on) with and without monitoring to assess the impact of code instrumentation on the runtime behaviour of a given task set.
The code instrumentation and WCET estimation should be performed in a way that clarifies the contribution of each type of monitoring data (e.g., metrics, logs, traces/spans) to the WCET calculation.
Suggestion: The OpenTelemetry (OTel) framework also provides a sort of hooking features, allowing to automatically generate monitoring data. An additional evaluation of this technique, compared to explicit code instrumentation is advised.
We suggest the following real-time workloads: *RT-APP*, *Cyclictest*, or any real-time workload provided during the course.
Important Note: The OpenTelemetry Framework does not support the *C language*. Thus, it is important for the students to choose a real-time workload written in supported Languages. If it is feasible, students may re-implement the task in languages that support both *libc* (to employ *RT-POSIX* libraries) and the OTel libraries, e.g., *Rust*. *RT-APP* and *Cyclictest* are implemented in *C*.

- rt-app: https://github.com/scheduler-tools/rt-app
- cyclictest: https://wiki.linuxfoundation.org/realtime/documentation/howto/tools/cyclictest/start

Tasks for Students
Discuss the possibility of leveraging these techniques to improve real-time system performance. In particular, the students should be able to answer the following research questions:

1. Given the tasks' deadlines, does the monitoring affect the ability of the system to remain compliant with its timely specification?
2. Can we collect relevant data to drive real-time behaviour of each task? (i.e., if the monitoring data hints at a certain task overloading the network, or requiring excessive time to respond to a request, can we perform a task eviction in a timely manner to ensure the remaining tasks stay compliant?)

Students are invited to perform an experimental campaign as they deem appropriate in order to answer the research question at the best of their capabilities.

What the Students Will Learn

- Dealing with real, state-of-art monitoring tools and real-time tasks and constraints.
- Understanding of the impact of monitoring overhead on SLOs compliance.
- Understanding the feasibility of leveraging specific monitoring data to perform online schedulability checks.

Required Tools

- A real-time OS (e.g., PREEMPT-RT Linux), deployed as bare-metal.
- RT-POSIX.
- OpenTelemetry Framework.

Expected Deliverables

- Source code and configuration files.
- Students are invited to report any kind of data to support their experimental campaign.
- Remember to maintain statistical relevance when acquiring any kind of measurement, performing experiments 10-30 times. This ensures that experimental evaluations are not misled by stochastic events.

Team size: 1-2 person/people. Duration: 4 weeks

A5) Assessing the OpenTelemetry (OTel) capabilities for mixed-criticality workloads.

Monitoring is a crucial step of a system operation, especially in real-time scenarios. During this process, an important step is to identify the monitoring data to produce, employing the proper frameworks and technologies to gather and analyze the system while maintaining its nominal operational behavior.
The OpenTelemetry framework has been widely adopted in recent years due to its versatility and applicability in several steps of the monitoring pipeline, from the production of monitoring data to its collection, processing and visualization. Mixed-criticality systems are characterized by a collection of real-time tasks with different criticalities/priorities. Monitoring infrastructures adopted for these platforms should be able to prioritize critical tasks over low-criticality or best-effort ones.

The students are asked to set up a simple mixed-criticality workload to execute and monitor. Students should be able to assess the already available OTel capabilities (e.g., tracing volume, sampling rate, etc.) to prioritize high-critical tasks. Students should also evaluate eventual monitoring overhead on the WCET.
Suggestion: OTel does not support the *C language*; students can write their real-time tasks in any of the languages supported by the Framework (check the official documentation). *Rust* is supported and can be employed to leverage both the *libc* and OTel libraries. When developing mixed-criticality workloads, students can reproduce

or instrument open-source tasks employed in the state-of-the-art or create their own (keep in mind that the main focus of this project is to evaluate the OTel capabilities for mixed-criticality systems, and to develop a complex mixed-criticality task set).

Tasks for Students
Students should set up an experimental campaign to provide evidence of actual task prioritization. If the results of such evaluation should provide evidence of inefficient or non-existent OTel capabilities, students should analyze the OTel codebase to identify potential improvements to its architecture.

What the Students Will Learn
- Dealing with real, state-of-art monitoring tools and real-time tasks and constraints.
- Understanding of the impact of monitoring overhead on SLOs compliance.
- Understanding the different configurations of the OpenTelemetry Monitoring Framework in the context of mixed-criticality systems.

Required Tools
- A real-time OS (e.g., PREEMPT-RT Linux), deployed as bare-metal.
- RT-POSIX
- OpenTelemetry Framework.

Expected Deliverables
- Source code and configuration files.
- Students are invited to report any kind of data to support their experimental campaign.
- Remember to maintain statistical relevance when acquiring any kind of measurement, performing experiments 10-30 times. This ensures that experimental evaluations are not misled by stochastic events.

Team size: 1-2 person/people Duration: 4 weeks

A6) Xen vs KVM under Mixed-Criticality Workloads: an Experimental Characterization

This project characterizes how the two general-purpose hypervisors covered in the course, **Xen** and **KVM**, behave when used to host **mixed-criticality** workloads on commodity hardware. The students design and run a measurement campaign on both hypervisors with the same workloads and stressors, quantifying the impact of the main isolation mechanisms available on each platform (vCPU pinning, host-side `isolcpus` / `nohz_full`, IRQ steering, cache and memory-bandwidth partitioning via Intel RDT) and of the **choice of guest type** (HVM, PV, PVH for Xen; full Linux guest vs **Unikraft** unikernel for both hypervisors).

The project takes the experimental analyses of Abeni and Faggioli (2019, 2020, see **Reading Material** below) as both a methodological anchor and a starting point. Those papers are now five years old and were performed on Linux 4.x/5.0 kernels, Xen 4.11, and pre-2020 hardware. Part of the project is to reproduce their main findings on the current software stack (Linux 6.x, mainline PREEMPT_RT, recent Xen, modern Intel/AMD CPUs) and to extend the analysis along dimensions the original work did not cover: real benchmark workloads (TACLe), memory-bandwidth partitioning, and unikernels.

Tasks for Students

The tasks are organized in three blocks:

1. **Baseline campaign and reproduction**
    1. Set up Xen and KVM on the same host. Define a single Linux guest configuration (vCPUs, memory, PREEMPT_RT kernel; HVM for Xen at this stage) and replicate it on both hypervisors.
    2. Reproduce the key experiments of Abeni and Faggioli (2019, 2020) on the current software stack:
        - cyclictest in the guest with various host/guest kernel combinations (RT and non-RT), with and without a background stress workload in Dom0 / on the KVM host;
        - effect of the QEMU Device Model priority on Xen HVM (the priority-inversion experiment of Section 6.2 of the 2020 paper);
        - effect of the Xen scheduler choice, using `null` as a counterpart to the default Credit scheduler. The goal is to confirm or update the published findings on modern systems. The `TIMER_SLOP` issue should already be fixed since Linux 5.2; the HVM priority inversion via QEMU may still be present. Different CPU generations are known to produce different absolute numbers (notably due to rdtsc emulation), so any deviation from the original numbers must be interpreted in this light.
    3. Extend the analysis to a real benchmark workload: **TACLe Benchmarks** (https://github.com/tacle/tacle-bench) running in a "critical" guest, with **stress-ng** and other stressors running in a co-located "noisy" guest.
    4. Apply the simplest isolation methods (vCPU pinning, `isolcpus`, IRQ steering) and report all results with appropriate visualizations.
2. **Guest-type comparison**
    1. For Xen, extend the study to all three guest types: **HVM**, **PV**, **PVH**. The Abeni/Faggioli work shows that this choice has dramatic effects on latency, so it is treated as a first-class dimension of the analysis.
    2. Run the campaign for each isolation method individually and for meaningful combinations, identifying which of them makes the biggest difference, on which hypervisor and guest type, and why.
3. **Unikraft unikernels as alternative guests**
    1. Build a **Unikraft** unikernel (https://unikraft.org/) hosting the critical workload (TACLe Benchmarks on top of Unikraft's minimal scheduler). Boot it as a guest on both KVM and Xen.

2. Compare the unikernel guest against the full Linux PREEMPT_RT guest in terms of: baseline latency, sensitivity to interference from the noisy neighbor, boot time, and resource footprint.

What the Students Will Learn

- A practical, comparative understanding of two different hypervisor architectures (type-1 vs type-2), beyond their theoretical presentation in the course.
- The Linux-side isolation primitives covered in the course are applied in a real measurement context.
- Experimental methodology for performance characterization of virtualized real-time systems: scenario design, statistical analysis, reproducibility, comparison against published baselines.
- (Block 3) Unikernel concepts and their relevance for specialized, predictable execution environments.

Reading Material

The two papers by Abeni and Faggioli below are the methodological anchor of this project. Students are expected to read them and to design the experimental campaign by direct comparison with their setup:

- L. Abeni, D. Faggioli, *An Experimental Analysis of the Xen and KVM Latencies*, IEEE ISORC 2019. DOI: 10.1109/ISORC.2019.00014
- L. Abeni, D. Faggioli, *Using Xen and KVM as real-time hypervisors*, Journal of Systems Architecture, vol. 106, 2020. DOI: 10.1016/j.sysarc.2020.101709

For Block 3, the Unikraft documentation (https://unikraft.org/docs) and the EuroSys 2021 paper *"Unikraft: Fast, Specialized Unikernels the Easy Way"* by Kuenzer et al. are the natural starting points.

Required Tools

- A development PC with Intel virtualization support (VT-x) and modern RDT features.
- Standard Linux tooling, Xen, KVM, cyclictest, TACLe Benchmarks, stress-ng, memguard.
- The Unikraft build environment.

Expected Deliverables

- The full measurement infrastructure (configurations, scripts, raw data) in a self-contained repository.
- An experimental report including methodology, results, statistical analysis, discussion, and an explicit comparison with the Abeni/Faggioli findings.

Team size: 1-3 people, Duration: 4 weeks

A7) Can Real-Time Linux Compete with FPGAs for Control Applications?

Modern control systems, such as those used in autonomous vehicles, spacecraft, and industrial automation, require fast and deterministic real-time performance. FPGAs offer parallel execution and precise timing behaviour, making them well-suited for such applications. However, FPGA-based development has traditionally required specialist hardware design expertise and relatively high development time.

Model-based design techniques have mitigated these limitations by enabling control algorithms to be designed and validated at a high level in simulation and then automatically translated into hardware implementations through code generation, reducing both development complexity and time. Even so, FPGA-based approaches still have several disadvantages, including cost, limited flexibility, and increased integration and debugging effort compared to software-based solutions, which limits their broader adoption despite their suitability for many application domains.

To address these limitations, software-based approaches for achieving real-time performance on general-purpose processors have evolved significantly. On one hand, there are dedicated solutions specifically designed with real-time guarantees in mind, such as real-time operating systems like VxWorks or PikeOS. On the other hand, the demand to combine real-time capabilities with a convenient, flexible, and rich software ecosystem has driven the development of Linux-based approaches. As a result, solutions like the PREEMPT_RT patch, resource isolation mechanisms like KVM-based virtualization, or resource regulation mechanisms like MemGuard have been introduced to improve predictability, while retaining the advantages of Linux.

The above raises an interesting question: can Linux-based solutions provide sufficient determinism for control workloads while enabling the flexibility and consolidation advantages unavailable to FPGA-based solutions?

The objective of this project is to compare the real-time performance of a control loop implemented on both an FPGA and a Linux-based real-time environment leveraging a model-based approach. Starting from an existing control algorithm developed in MATLAB/Simulink, the HDL code and the C code of the control loop are automatically generated and then deployed onto their respective target platforms for benchmarking.

Experimental Setup

The systems considered are the programmable logic (FPGA) on a Zynq Ultrascale+ (ZCU102) and a bare-metal real-time Linux environment in 4 different configurations:

- **Configuration 1:** Linux is patched with PREEMPT_RT and the control loop runs without particular provisions.

- **Configuration 2:** Linux is patched with PREEMPT_RT and the control loop runs on an isolated CPU (cpu affinity, isolcpus, nohz_full, rcu_nocbs).
- **Configuration 3:** Linux is patched with PREEMPT_RT, the control loop runs on an isolated CPU (cpu affinity, isolcpus, nohz_full, rcu_nocbs) in the presence of an untrusted “noisy neighbor” running on KVM
- **Configuration 4:** Linux is patched with PREEMPT_RT, the control loop runs on an isolated CPU (cpu affinity, isolcpus, nohz_full, rcu_nocbs) in the presence of an untrusted “noisy neighbor” running on KVM, and MemGuard is used to regulate memory bandwidth.

The “noisy neighbor” can be any stressor of choice (e.g., stress-ng) and must target both CPU and memory. The configurations are summarized in the table below:

Configurations	PREEMPT_RT	CPU Isolation	Stressor on KVM	MemGuard
Configuration 1	✓			
Configuration 2	✓	✓		
Configuration 3	✓	✓	✓	
Configuration 4	✓	✓	✓	✓

The metrics of interest are three:

- **Jitter:** the deviation from nominal periodicity of the control loop, or in other words the variation in time between when a control event is expected to happen and when it actually happens
- **Deadline Miss Ratio (DMR):** the percentage of missed deadlines computed as DMR=missed deadlines/total activations
- **Execution Time Variability:** the degree to which a task’s execution time fluctuates across repeated runs under identical conditions.

The control loop is evaluated at 1 kHz, 5 kHz, and 10 kHz. The experiments are summarized in the table below:

Experiment	Target	Configuration	Loop Rate
Baseline A	**FPGA**	**–**	**1 kHz, 5 kHz, 10 kHz**
Baseline B	**RT-Linux**	**Configuration 1**	**1 kHz, 5 kHz, 10 kHz**
Test 1: CPU Isolation	**RT-Linux**	**Configuration 2**	**1 kHz, 5 kHz, 10 kHz**
Test 2: Contention	**RT-Linux**	**Configuration 3**	**1 kHz, 5 kHz, 10 kHz**
Test 3: Regulation	**RT-Linux**	**Configuration 4**	**1 kHz, 5 kHz, 10 kHz**

Other parameters such as duration, repetition and so on can be chosen freely.

Tasks for Students

Phase 1: Project Foundations

- Study how to generate C code for embedded targets using Embedded Coder in Simulink.
- Study how to generate VHDL/Verilog code for HDL targets using HDL Coder in Simulink and how to target the AMD ZCU102 board specifically.
- Adapt the existing control algorithm in MATLAB/Simulink, ensuring it is strictly discrete-time and compatible with both Embedded Coder (for C code generation) and HDL Coder (for HDL code generation).

Phase 2: FPGA

- Set up the ZCU102 and deploy an example IP using HDL coder to confirm the toolchain and board are functioning correctly.
- Use HDL Coder to deploy the control loop onto the ZCU102 and confirm it is working correctly.
- Implement non-invasive instrumentation and run the experiments.
- Collect and analyse execution metrics.

Phase 3: Real-Time Linux

- Configure a **bare-metal** Linux environment as per experimental setup.
- Use Embedded Coder to generate the C code for the control loop, compile it, and deploy it on the system.
- Once the setup is working correctly, implement non-invasive instrumentation and run the experiments.
- Collect and analyse execution metrics.

Phase 4: Comparative Analysis & Critical Evaluation

- Compare the data and evaluate how real-time Linux performs compared to the FPGA.
- Evaluate whether the proposed Linux configuration is the optimal solution for a control loop and discuss alternative solutions.
- **(Optional)** Perform a Design of Experiments (DoE) to evaluate the impact of PREEMPT_RT, CPU isolation, and MemGuard on real-time performance metrics (execution time variability, jitter, and deadline miss ratio)

Learning Outcomes

- How real-time control systems can be implemented on both FPGA and Linux-based platforms using model-based design tools.

- First-hand experience of the trade-offs between hardware-based (FPGA) and software-based real-time implementations in terms of determinism, flexibility, and complexity.
- How interference and resource contention affect the behaviour of real-time control loops.
- How to evaluate real-time performance using practical metrics such as execution time variability, jitter, and deadline miss ratio.
- How to critically assess the suitability of different computing platforms for real-time control applications based on experimental evidence.

Required Tools

- AMD ZCU102 evaluation board.
- Development PC with **bare-metal** linux.
- Development PC with MATLAB/Simulink 2024b or 2025b.

Expected Deliverables

- GitHub repository containing all source code, build files, configuration artifacts, and any additional resources required to reproduce the experiment.
- Technical documentation within the repository, including setup instructions, usage guide, and implementation notes.
- Report containing the methodology and the results of the performed analyses

Team Size: 1-2 people, Duration: 4 weeks

A8) Experimental Evaluation of Edge AI Inference for UAV Agentic Applications

AI-enabled UAV systems require low-latency, autonomous decision-making under strict SWaP (size, weight, and power) constraints. A key architectural question is whether AI inference should be performed onboard the UAV or offloaded to a Ground Control Station (GCS). This design choice directly affects latency, robustness to communication delays, and overall system responsiveness. Rigorous benchmarking is therefore essential to justify and guide this architectural decision

The objective of this project is to assess the trade-offs between onboard and GCS-based AI inference and determine which architecture better satisfies the requirements of latency-sensitive UAV applications. To support this, an experimental evaluation framework will be designed and implemented to compare both approaches for a UAV-relevant task (e.g. object detection), using an embedded edge platform (e.g. AMD Zynq UltraScale+) as the onboard compute node and a Linux-based workstation as the GCS. The framework will evaluate both architectures under controlled conditions,

measuring inference latency and system-level response time across different operational scenarios, including varying communication conditions.

Tasks for Students

1. Study the system architecture of edge AI deployments for UAV applications, focusing on the trade-offs between onboard inference and offloaded inference to a Ground Control Station (GCS).
2. Set up an edge platform (e.g. AMD Zynq UltraScale+) and a Linux-based workstation to act as the onboard and GCS nodes respectively.
3. Implement an AI inference pipeline (e.g. object detection) that can run both locally on the edge device and remotely on the GCS.
4. Implement a communication mechanism between onboard and GCS systems to transmit sensor data and inference results under realistic network conditions.
5. Design and implement an experimental evaluation setup to test both inference architectures under controlled and repeatable conditions.
6. Introduce and vary communication conditions (e.g. latency, bandwidth constraints, or packet loss) to simulate realistic UAV operational environments.
7. Collect and analyse performance metrics, including inference latency and end-to-end system response time.
8. Compare onboard and offloaded inference approaches and interpret the results in terms of suitability for latency-sensitive UAV applications.
9. Document system design, implementation decisions, and experimental methodology in a structured technical report.

Learning Outcomes

- Hands-on experience with AI model deployment on edge platforms
- Practical understanding of the tradeoffs between on-board and offloaded AI inference in terms of latency, throughput, power consumption, and link reliability
- Familiarity with benchmarking methodologies and performance characterization techniques for AI inference systems
- Practical understanding of communication constraints and their impact on distributed AI pipelines in UAV scenarios

Required Tools

- Development PC with at least 150 GiB of available disk space
- AMD ZCU102 or AMD ZCU104

Expected Deliverables

- GitHub repository containing all source code, build files, configuration artifacts, and any additional resources required to reproduce the system.
- Technical documentation within the repository, including setup instructions, usage guide, and implementation notes.
- Technical report describing the system architecture, design decisions, and implementation choices
- Report containing the methodology and the results of the performed analyses

Team Size: 1-2 people, Duration: 5-6 weeks

A9) Fault Injection on the AI Accelerator Driver

On an edge device the real engine of language-model inference is the AI accelerator (GPU, NPU, TPU, or DPU). Its kernel driver receives commands from the inference runtime, manages data transfers to and from device memory, submits compute kernels, and collects results. It is therefore a critical, high-interaction component throughout inference, and it can fail due to soft errors, latent bugs, targeted attacks, or stress conditions. The research question is: when a fault occurs in the accelerator driver while the model is performing inference, does the model output change, and how? The answer is not obvious, because it depends on three dimensions, the accelerator, the driver point hit, and the inference phase in which the fault manifests, which the project characterizes experimentally. The literature on fault propagation in neural models has focused mostly on data (weights, activations); the accelerator driver is still poorly studied as a fault surface, especially when coupled with an LLM/SLM workload.

This is fault injection in the strict sense: the fault model is explicit (wrong return value of a driver function, delay, corruption of a command descriptor or buffer, failed ioctl), the injection points are known and deterministic, and the campaign is a controlled Design of Experiments.

Tasks for Students

- Study the assigned board's AI accelerator driver and identify the relevant injection points (ioctl handlers, command-queue management, DMA transfers, completion paths).
- Build a fault-injection pipeline targeting the accelerator driver, based on the Linux kernel fault-injection framework (debugfs/sysfs, requires CONFIG_FAULT_INJECTION) as a baseline, integrated with eBPF/kprobe hooks for surgical injection on specific driver functions

(hooking the return of those functions and altering the value or introducing delays).

- Instrument the inference runtime with application-level phase hooks so that injections can be synchronized with the inference phases, model load onto the accelerator, prefill, decode, allowing statements such as “this fault was injected during the decode of token N”, which is crucial for propagation analysis.
- Design a variable workload along two axes: model scale (SLM 1–3 B; LLM 7–13 B where the platform allows) and task type (arithmetic, multiple-choice QA, structured extraction).
- Run a stratified experimental campaign over driver points × inference phases × model, classifying every run with the outcome taxonomy (crash, hang, Silent Data Corruption, benign) and measuring semantic degradation with objective metrics (accuracy drop, SDC rate, crash/hang rate). Use greedy decoding with fixed seeds.
- Synthesize the results into a joint accelerator/driver-point/phase/model-scale vulnerability characterization, highlighting which combinations are most vulnerable and whether larger LLMs are more resilient (parametric redundancy) or more fragile (longer temporal exposure) than SLMs.
- (Optional, for larger groups) Extend the campaign to a second board to compare how the same fault model propagates across different accelerators and drivers.

What the Students Will Learn

- The internal interaction between an inference runtime and an AI accelerator driver, from ioctl submission to DMA completion.
- Practical use of the Linux kernel fault-injection framework and of eBPF/kprobe for targeted kernel-level fault injection.
- The notion of fault propagation and the distinction between crash, hang, and Silent Data Corruption in a non-deterministic ML workload.
- Methodology to design a controlled, reproducible fault-injection campaign and to analyze its results statistically.

Required Tools

- OrangePi 5 plus, NVIDIA Jetson Orin, Beaglebon, AMD ZCUs (104, 102, Krya) with a kernel rebuilt to enable CONFIG_FAULT_INJECTION and eBPF/kprobe on the accelerator-driver functions.
- The inference runtime that actually uses the board accelerator (llama.cpp with the appropriate backend where available, otherwise the accelerator’s native stack: RKNN/RKLLM, libedgetpu, Vitis-AI).
- SLM models in GGUF; optionally an LLM where the platform allows.

- eBPF tooling (bcc/libbpf), and a fast filesystem-rollback procedure (snapshot) for runs that leave the system unstable.

Expected Deliverables

- Source code of the fault-injection pipeline and experimental harness, in a reproducible repository.
- The dataset of raw and aggregated results.
- An experimental report with methodology and the accelerator × driver-point × phase × model-scale propagation analysis, including visualizations.

Notes

Team size: 1–3 people (project scope scales with team size; the optional tasks are expected for larger groups, as agreed with the tutor). Duration: 4 weeks..

A10) Fuzzing the AI Accelerator Driver During Inference

The AI accelerator driver exposes a user-space interface, mainly through ioctls on the accelerator device file, plus mmap and related syscalls, through which the inference runtime submits commands, allocates device memory, launches compute kernels, and retrieves results. This interface is a typically complex and less-exercised error surface: edge accelerator drivers are often relatively young, vendor-developed, and handle intricate data structures (command descriptors, memory maps, queues). Fuzzing is the standard technique to empirically discover inputs and sequences that cause crashes, hangs, deadlocks, memory corruption, or anomalous driver states. Here nothing to break is decided in advance.

The project's research question is twofold: which inputs to the accelerator driver reveal its fragility, and *how does the error surface change when a language-model inference process runs concurrently*, populating the driver with real state (allocated contexts, active queues, device memory occupied by the model weights)? This is fuzzing in the strict sense: the "bug" is any anomalous driver behavior the fuzzer manages to trigger, and discovery is the product of the project.

Tasks for Students

- Set up syzkaller, the de facto standard for coverage-guided Linux-kernel fuzzing, on the assigned board, with a kernel rebuilt with KCOV, KCOV_INSTRUMENT_ALL, DEBUG_FS, and the KASAN/UBSAN sanitizers (golden image prepared by the tutor).
- Write/adapt the syzkaller interface descriptions (the .txt files describing ioctls, data structures, and the device file) for the assigned board's

accelerator driver — the qualifying part of the work, which turns syzkaller from a generic kernel fuzzer into a targeted accelerator-driver fuzzer.

- Design an experimental scenario in which fuzzing runs while an inference process is using the accelerator on the same board. This requires syzkaller's *isolated* mode (fuzzing real hardware over SSH) rather than QEMU mode, also necessary because the physical accelerator is not present in a VM, and synchronization between fuzzer start and inference start.
- Define a bug oracle at the kernel level (panic, oops, warning, deadlock detection, KASAN/UBSAN reports, hang) and at the application level (crash or stall of the inference process, degenerated output).
- Compare the bugs and driver code paths reached by fuzzing in three conditions: (a) baseline with no workload, (b) with an SLM running inference, (c) with an LLM running inference where the platform allows. Analyze the differential coverage: are there driver paths reached only in the presence of the inference workload?
- For each bug, assess the root cause (driver function involved) and reproducibility, and classify the failure modes.
- (Optional, for larger groups) Develop a custom mutation script that generates calls targeting the specific ioctl patterns emitted by the runtime during inference (device-memory allocation, command submission, synchronization), to explore paths syzkaller reaches with low probability through its generic mutation strategy.

What the Students Will Learn

- The internals of coverage-guided kernel fuzzing and the syzkaller architecture (descriptions, manager, executor, KCOV-based coverage).
- How an AI accelerator driver exposes its interface to user space and how to model it for a fuzzer.
- Bug triage methodology: deduplication, minimization, root-cause analysis, and the use of kernel sanitizers.
- How a concurrent real workload changes the observable error surface of a driver

Required Tools

- One edge board from the available pool, provided by the tutor, with a kernel rebuilt with KCOV/KASAN (golden image prepared by the tutor); a board with a well-documented open-source kernel driver is strongly preferable for this project.
- syzkaller (Go toolchain) and the inference runtime that uses the board accelerator.
- SLM models; optionally an LLM where the platform allows.

- A fast rollback procedure and watchdog/timeout handling for fuzzing on real hardware.

Expected Deliverables

- The syzkaller interface descriptions for the accelerator driver and the fuzzing configuration for the target board.
- The orchestrated experimental scenario (campaign scripts) and any custom mutation scripts.
- The dataset of crash reports and minimized cases.
- An experimental report with the catalogue of discovered failure modes and the comparative baseline / SLM / LLM coverage analysis.

Notes

Team size: 1–3 people (project scope scales with team size; the optional tasks are expected for larger groups, as agreed with the tutor). Duration: 4 weeks.

Implementation Projects

I1) Edge AI on Embedded Systems Synthesis

This project explores the deployment and evaluation of multiple lightweight AI models on embedded edge devices such as the Google Coral Dev Board, BeagleBone AI, Orange PI Plus 5. The goal is to investigate how different types of models perform when run on-device, particularly in real-time applications that demand low latency and isolation from system-level interference.

Students will deploy, run, and analyze the following AI models:

- Small language model – to test the behavior of an AI agent deployed at the edge (e.g., Ministral, Qwen3.5-0.8B, etc.)
- Semantic Segmentation – Label each pixel with a class to distinguish different regions or objects (e.g., DeepLab).
- Audio Classification – Classify sound events or speech commands using onboard microphones (e.g., YamNet or Speech Command models).

A critical part of the assignment is to evaluate the impact of system interference (e.g., logging, I/O, networking, visualization) on model performance and to explore isolation techniques that ensure stable, real-time inference.

Tasks for Students

1. **Hardware and Environment Setup**
    a. Configure the Google Coral or BeagleBone AI or OrangePI development board with necessary AI runtimes (e.g., TensorFlow Lite, Edge TPU, OpenCV, audio libraries).
    b. Test camera and microphone input pipelines for vision and audio models.
2. **Model Deployment**
    a. Load and run one of the following models:
        i. **Small language model**: mimicking a local Ai agent
        ii. **Semantic Segmentation**: Apply segmentation to live camera input to identify regions like roads, people, or furniture.
        iii. **Audio Classification**: Classify environmental sounds or speech commands using the onboard mic.
    b. Measure baseline inference time for the chosen model.
3. **Isolation and Interference Evaluation**
    a. Introduce controlled interference scenarios:
        i. Simultaneous logging, file I/O, camera recording, network streaming, or background processes.
    b. Measure performance degradation: latency increase, detection/classification accuracy, generated text quality
    c. Implement isolation strategies:
        i. CPU core pinning or process isolation
        ii. Prioritized scheduling or async inference
        iii. Splitting I/O and inference into separate threads or containers (if supported)
        iv. Use hypervisor-based solutions (e.g., KVM or Jailhouse)
    d. Repeat measurements to evaluate improvements with isolation.
4. **Real-Time Performance Testing**

a. Create test loops for each model where low latency is critical (e.g., detecting a spoken command and responding within 300 ms, tracking hand movement at 30 FPS).

b. Validate the real-time behavior of models with and without isolation mechanisms.

5. **Reporting and Demonstration**

a. Document results, showing:

i. Performance under interference

ii. Impact of isolation strategies

iii. Trade-offs in resource usage across models

b. Present visualizations (graphs, tables) of key metrics.

c. Record or live-demo the system running selected models in a real-time scenario.

What the Students Will Learn

Students will gain experience in deploying and running a variety of AI models across multiple domains (vision and audio) on top of constrained hardware (edge AI). Learn to identify how background system activity affects inference speed, accuracy, and responsiveness in real-time applications and explore practical techniques to isolate inference tasks for minimal latency and high reliability.

Notes

Team size: 1–3 people (project scope scales with team size; the optional tasks are expected for larger groups, as agreed with the tutor). Duration: 4 weeks.

I2) Non-intrusive mixed-criticality RT-monitoring

The idea underlying this project is to design and prototype a non-intrusive monitoring solution that meets the requirements of real-time mixed-criticality systems, without interfering with the tasks being monitored. To this aim, the students will study and analyze technologies underlying direct monitoring solutions and leverage one (or more) of them to design a prototypal monitoring approach. The proposal will be then evaluated and compared with some existing non-RT monitoring solutions.

Tasks for Students

1. **Environment Setup**
    a. Prepare an environment with Zephyr
    b. Deploy RT mixed-criticality workload, e.g., rt-app, cyclictest, or custom rt-application for Zephyr

2. **Design and prototype**
    a. Explore existing distributed tracing solutions, like OpenTelemetry
    b. Design a prototypal solution that mimics the explored technology by using RT-POSIX API and considering the mixed-criticality scenario
    c. Evaluate the failure reporting ability of the proposal under different failure conditions.

3. **Measurements**
    a. Measure baseline performance (e.g., latency) of workloads without monitoring.
    b. Measure the performance of workload when the monitoring is enabled, under different monitoring conditions, such as varying monitoring period, number of data sources, and running the monitoring on isolated cores (e.g., CPU core pinning or process isolation).
    c. Evaluate its ability to prioritize the monitoring of high-critical tasks over less critical ones.
    d. Compare the prototype solution with existing monitoring solutions in terms of overhead, interference, and capabilities.

4. **Reporting and Demonstration**
    a. Document results, showing:
        i. Performance without monitoring
        ii. Impact of monitoring solution/s in the different considered conditions
        iii. Capabilities of the solution in the context of mixed-criticality systems
    b. Present visualizations (graphs, tables) of key metrics.

Team size: 1–3 people (project scope scales with team size; the optional tasks are expected for larger groups, as agreed with the tutor). Duration: 4 weeks.

I3) Extending runPHI with a KVM Backend

runPHI is an OCI-compliant container runtime, written in Rust, that integrates hypervisors with standard container orchestration systems (e.g., Kubernetes). It enables the deployment of what we call **partitioned containers** (**p-containers)** when a partitioning hypervisor is used as the backend, and **Xen containers** when the

backend is Xen. They are guest VMs but seen and managed as containers that run RTOS or bare-metal workloads on industrial hardware platforms with strong temporal and spatial isolation guarantees. The runtime currently supports two backends: **Jailhouse** and **Xen**.

runPHI is evolving towards a more general role as a **meta-runtime and orchestration companion** able to seamlessly manage heterogeneous container workloads across multiple virtualization technologies, with different trade-offs in terms of dependability, performance, and isolation, which in a mixed-criticality scenario is exactly the core problem to address.

In this project, the students will extend runPHI by implementing a new **KVM backend**. KVM is the de facto standard hypervisor in the Linux world and is part of the mainline kernel; integrating it into runPHI significantly broadens the range of platforms, guest types, and deployment scenarios the runtime can support. Unlike the currently supported partitioning hypervisors, KVM does not provide isolation guarantees comparable to those of p-containers out of the box, which makes the design and the experimental evaluation of the integration particularly interesting.

Tasks for Students

The tasks are organized in three blocks. **Block 1 is mandatory for all groups.** For groups of two students, **Block 2 or Block 3** is also expected (the choice will be agreed with the tutor). For groups of three students, **both Block 2 and Block 3** are expected.

1. Implementation of the KVM backend (mandatory)

- Study the runPHI architecture, the backend abstraction layer, and the existing Jailhouse and Xen backends to understand the contract a backend must implement.
- Implement the new backend in runphi_manager (Rust), keeping it architecture-independent wherever possible. The backend must support at least the basic OCI lifecycle calls: create, start, stop, kill, and delete. KVM also enables operations that the partitioning-hypervisor backends do not expose well (e.g., pause/resume, dynamic resource update, or checkpoint/restore via QEMU snapshots): supporting any of these as additional OCI calls is encouraged but optional.
- Use a **Zephyr guest** as the default reference workload; a **bare-metal** guest is an acceptable alternative.
- Containerize a small validation application of your choice and contribute it to the partitioned_container_demos repository as part of the deliverables.
- *(Optional, recommended for larger groups)* Extend environment_builder to automate the KVM backend setup. The repository is designed for setups that are non-trivial to reproduce: the x86 setup is simple enough not to warrant automation, whereas an ARM board target can benefit from it to improve

reproducibility.

2. Isolation techniques on top of KVM (extension)

- **Implement** isolation techniques already covered in the course: vCPU pinning, host-side isolcpus / nohz_full, IRQ steering, and memory locking.
- **Investigate and then implement** isolation techniques that are only marginally touched in the course: detailed **cgroups** configuration, **memory-bandwidth partitioning** (software, via **memguard**, and hardware, via Intel RDT MBA on x86 or ARM MPAM where available), **cache partitioning** (Intel CAT, ARM MPAM), and **device passthrough via VFIO/IOMMU**.
- Integrate the relevant techniques into the KVM backend so they can be enabled per-container via runPHI's configuration mechanisms, extending those mechanisms as **needed**.

3. Comparative evaluation (extension)

- Design an experimental campaign comparing the KVM backend against the other backends already supported on a comparable hardware/guest combination.
- Relevant metrics include container start/stop latency, steady-state jitter within the guest (e.g., cyclictest-style measurements), and performance degradation under controlled interference from a noisy Linux co-tenant.
- Repeat each measurement about 30 times to ensure statistical relevance, and report the results with appropriate visualizations.

What the Students Will Learn

- The internals of an OCI container runtime and how higher-level tools like **Docker** interact with it.
- The architectural differences between **partitioning and fully type 1 hypervisors** (Jailhouse, Xen) and a **type-1 hybrid hypervisor** like KVM, and the implications of these differences for mixed-criticality workloads.
- Practical **Rust** development on a real, actively maintained research codebase. Prior Rust knowledge is helpful but not required; the project's main task is Rust programming, which will provide a substantial learning opportunity.
- (Block 2) How to **apply and deepen** the use of Linux-based isolation primitives.
- (Block 3) Methodology to design a reproducible experimental campaign on virtualized real-time systems.

Required Tools

- A development PC with KVM support (any modern x86 CPU with VT-x/AMD-V).
- A **Xilinx ARM board** (ZCU104/102 or KV260/KR260) provided by the tutor for ARM-target experiments (mainly for larger groups).
- The runPHI codebase: runphi_manager, environment_builder, partitioned_container_demos (https://github.com/runphi).

- A Zephyr toolchain (or bare-metal toolchain) for the guest workloads.

Expected Deliverables

- Source code of the new KVM backend, contributed as a fork/branch of runphi_manager (ideally in a form suitable for upstream merge).
- A validation application, containerized and added to partitioned_container_demos.
- *(Optional)* Configuration templates and automation in environment_builder.
- (Block 3) An experimental report including configurations, raw and aggregated measurements, and a discussion of the results.

Team size: 1-3 people, Duration: 4 weeks

I4) Containerizing LinuxCNC as a Partitioned Container

LinuxCNC (https://linuxcnc.org/) is the open-source motion controller used in real CNC machine shops worldwide. It implements multi-kHz hard real-time control loops on top of a PREEMPT_RT Linux kernel, with direct access to motion-related I/O (steppers, encoders, limit switches, PWM). Its established industrial deployment makes it a credible test workload for studying the real-time guarantees that survive when an industrial control workload is containerized.

The project consists of containerizing LinuxCNC as a **partitioned container** on runPHI, executing it on a Linux guest with PREEMPT_RT via the **Xen** backend, and characterizing its real-time behavior end-to-end. The goal is twofold: (a) run a diffused industrial workload as a p-container, and (b) measure what real-time guarantees the partitioning preserves with respect to a bare-metal LinuxCNC baseline.

Tasks for Students

1. Study the runPHI architecture and the Xen backend, together with the LinuxCNC architecture (HAL, motion controller, latency requirements).
2. Build a runPHI partitioned container that encapsulates LinuxCNC on top of a PREEMPT_RT Linux guest. Run it on Xen.
3. Define a representative motion-control scenario (a simulated machining task that exercises the motion controller without requiring physical I/O) and run it both bare-metal (Linux PREEMPT_RT directly on the host) and inside the partitioned container.
4. Measure and compare the real-time behavior of the two configurations, focusing on the LinuxCNC latency-test results (the project's own diagnostic tool) and on missed-deadline rates of the motion controller. Repeat each measurement 10-30 times for statistical relevance.

5. Document the I/O exposure strategy. Even though the experimental campaign uses simulated I/O, the student must describe how real motion-control I/O would be exposed to the p-container (vCPU pinning, IRQ steering, IOMMU/passthrough considerations), since this is the actually-hard part of any real industrial deployment.

What the Student Will Learn

- The internals of an industrial-grade open-source motion controller (HAL architecture, real-time kernel modules, motion planning).
- The integration challenges of running an industrial Linux workload as a p-container, including the trade-offs between strict partitioning and the workload's I/O assumptions.
- Practical use of runPHI on the Xen backend with a Linux guest.
- Methodology for characterizing the real-time behavior of a containerized RT workload against a bare-metal baseline.

Required Tools

- A development PC able to run Xen with a Linux guest (Intel VT-x or AMD-V).
- The runPHI codebase: `runphi_manager`, `partitioned_container_demos` (https://github.com/runphi).
- LinuxCNC and its toolchain.

Expected Deliverables

- A partitioned container image of LinuxCNC, contributed to `partitioned_container_demos`.
- A short experimental report comparing bare-metal vs containerized LinuxCNC, including the I/O exposure discussion.

Team size: 1-2 persons, Duration: 4 weeks

I5) Design and Implementation of a CAN Bus Analyzer for System Monitoring and Diagnostics

Controller Area Network (CAN) buses are widely used in automotive and industrial systems for reliable, real-time communication, particularly in applications where deterministic message delivery and robustness are required under harsh operational conditions (e.g., EMI). A CAN bus analyzer provides insight into system behavior by capturing, decoding, and interpreting network traffic, enabling applications such as fault detection and intrusion detection, thereby ensuring safe and reliable operation.

In addition, CAN bus analyzers are also used for subsystem emulation as well as for reverse engineering of message formats and communication patterns.

The project consists of designing a CAN bus analyzer that decodes a custom higher-layer protocol and implementing it on the processing system (PS) or the programmable logic (PL) of an AMD Zynq UltraScale+. The analyzer is deployed in the flight control system of a UAV and is intended to identify devices currently communicating on the bus, monitor deviations in acquisition and actuation frequencies from nominal values, detect missing messages, and optionally support subsystem emulation.

Tasks for Students

1. Study the architecture of the AMD Zynq UltraScale+ MPSoC, focusing on PS–PL partitioning and communication, and how peripherals are connected, routed, and accessed across the system.
2. Study the architecture of the evaluation board (ZCU102 or ZCU104) to understand how the CAN peripheral is exposed, and determine what is required to establish CAN communication through either the PS or the PL using the available hardware interface.
3. Choose **one implementation approach** and implement basic CAN communication, i.e., message transmission and reception:
    - **PS track:** CAN communication is established through the PS
    - **PL track:** CAN communication is established through the PL
4. Develop a parser for the custom higher-layer protocol to interpret raw CAN frames and extract application-level messages. Detailed specifications of the higher-layer protocol will be provided at a later stage.
5. Build a data acquisition pipeline using the developed parser to extract diagnostic data required to identify devices currently communicating on the bus, monitor deviations in acquisition and actuation frequencies from nominal values, and detect missing messages. Detailed information regarding control system behavior will be provided at a later stage.
6. Perform online or offline analysis of the acquired diagnostic data to provide insight into system behavior
7. **(Optional)** Extend the developed analyzer to actively participate on the bus in order to emulate a device of choice

Learning Outcomes

- How to physically setup a CAN bus
- Hands-on experience with CAN communication.
- Hands-on experience with advanced features of MPSoC FPGA devices
- How to extract and interpret diagnostic data from bus traffic to infer system-level behaviour.
- Practical understanding of the need for higher-layer protocols on top of CAN

Required Tools

- AMD ZCU104 or AMD ZCU102 development board
- Development PC with Linux and at least 150 GiB of available disk space
- AMD PetaLinux tools and Vivado Design Suite

Expected Deliverables

- Working setup demonstrating basic CAN communication, protocol decoding, data acquisition, and data analysis.
- GitHub repository containing all source code, build files, configuration artifacts, and any additional resources required to reproduce the system.
- Technical documentation within the repository, including setup instructions, usage guide, and implementation notes.
- Technical report describing the system architecture, design decisions, and implementation choices

Notes:

- The PL track is mandatory for teams of 3 students or more

Team Size: 1-3 people, Duration: 4-5 weeks